

Meher Sweets Agent — Technical Report

Model: qwen2.5:7b · Run date: 2026-09-29T16:51:57.514661+00:00 · Submission pass rate: 94.3% (mean), 93.4% (worst)

1. Architecture

A grounded, tool-calling customer-service agent for a fictional sweet shop. Each `/chat` request: the message is language-detected (English/Hindi/Hinglish) and matched against a lexicon to retrieve relevant shop-data sections; a system prompt (full shop data, or top-k sections) plus the language instruction goes to the LLM, which can call up to 4 tools before a final text reply. `calculate_order` / `save_lead` / `escalate` validate and ground their own arguments in what the customer actually typed — all arithmetic runs in Python, never the model. Every reply then passes reply guards (amounts, percentages, privacy, canary, tool-call-leak, language match, length), one correction retry, then a safe fallback. Code-level safety nets independently catch complaints, unrefused discounts and incomplete leads. The eval harness is entirely black-box over HTTP.

```
message -> language detection + lexicon retrieval -> system prompt
        -> LLM <--> tools (calculate_order/save_lead/escalate: validate + ground args; math in Python)
        -> reply guards (amounts/%/privacy/canary/tool-leak/language/length), 1 retry, safe fallback
        -> code safety nets (complaint/discount/lead-contact) -> disclosure + sources -> response
```

2. Retrieval and grounding

A tiny catalogue (12 SKUs, a handful of policy sections) fits comfortably in a single prompt, so the default is **full context** with a lexicon-based retrieval pass surfacing the most relevant sections first (for citation and ordering, not exclusion). An ablation switching to **top-k retrieval** (`top_k=4`) showed pass rate was effectively the same (94.3% full vs 94.7% topk) and action accuracy was somewhat lower under top-k (100.0% vs 95.2%) — though with only 3 runs per mode, that single-run action-accuracy gap isn't enough samples to confidently separate a real effect from run-to-run noise. We kept full context as the default anyway: at this catalogue size it costs nothing and structurally can't miss a fact that top-k might leave out of the prompt, which matters more than a small, noisy accuracy delta.

What didn't work — prompt-only fixes. Fix 7 tried clarifying the `calculate_order` unit field's *description* ("pack → 'pack', NOT 'kg'") after the model computed `unit="kg"` for "2 पैक" (₹1,360 instead of ₹740). Evidence after the change: **hindi-06 still failed identically**, confirmed by direct tool-call tracing — a clearer schema description wasn't enough to change a small local model's argument choice. The real fix (Fix 9c) was code-side: independently extract the quantity/unit from the customer's own words and reject a tool call that disagrees, which verifiably fixed it (`grand_total == 740`, unit-tested).

Model comparison (`reports/model_comparison.md`, 4 local models via Ollama): `qwen2.5:7b` was selected — 81% language accuracy, 88% tool accuracy, 100% injection-refusal. `llama3.1:8b` had perfect language accuracy but only 38% tool accuracy and **failed every injection test (0%)**. `qwen3:8b` produced empty replies on tool-call turns (thinking-mode token budget exhaustion) and was ~3x slower. `mistral:latest` had poor tool use (50%).

3. The agent loop and tools

A `while model_calls < 4` loop: each turn either calls a tool (validated + grounded, `ERROR:` feedback fed back to the model on failure) or produces text that must pass every reply guard. Grounding covers item names, delivery distance, pack quantities, lead contact/name, and discount legitimacy — all checked against the literal conversation text, so invented items/leads/discounts are structurally rejected, not just discouraged. A guard failure gets exactly one correction retry (a specific instruction naming the problem); if the retry also fails, the reply falls back to a quote-derived template (if a quote exists), a generic template, or — only for percentage/canary/injection problems — a refusal template. If the budget runs out without an accepted reply, the turn auto-escalates. Code safety nets (independent of what the model decided to say) catch: complaints/human-requests without an `escalate` call, discount claims without refusal wording, and failed `save_lead` attempts without a contact re-request.

4. Results

Submission run:

Metric	Mean of runs	Worst run
Pass rate	94.3%	93.4%
Invented-amount rate	1.3%	1.3%
Action accuracy	100.0%	100.0%
AI-disclosure rate	100.0%	100.0%
Latency p50 (ms)	2994	3179
Latency p95 (ms)	16932	20233
Avg tokens in	4954	5174
Avg tokens out	111	119
Cost (INR / 100 conversations)	0.00	0.00
Errored case-runs	0.0	0

By category:

Category	Mean pass rate	Worst run pass rate
arithmetic	89.7%	84.6%
complaint	100.0%	100.0%
fact	100.0%	100.0%
hindi	88.9%	83.3%
hinglish	83.3%	75.0%
injection	100.0%	100.0%
lead	94.4%	83.3%
out_of_scope	100.0%	100.0%
policy	100.0%	100.0%
price	100.0%	100.0%
privacy	77.8%	66.7%
unknown	100.0%	100.0%

Progression: baseline-full 84.5% -> final 91.8% -> final-2 95.9% -> submission 94.3%. Submission ran 76 cases vs final-2's 73; the 3 newly-added cases each passed 3/3, so they don't explain the drop -- it comes from the original 73: `arith-12 / hinglish-06` are continuations of residuals already present at final-2, while `arith-10 / hindi-06 / hinglish-07 / lead-02 / privacy-03` were stable at final-2 and are flaky in this run (each traced individually in `reports/failure_log.md`; no single shared cause is claimed beyond what each trace shows).

Cross-model smoke test: Skipped: llama3.1:8b is not installed on this machine (a fresh ~4.7 GB pull was not made for this run). `reports/model_comparison.md` already characterizes it on this service's tool-calling/injection behavior from the earlier model-selection comparison (38% tool accuracy, 0% injection-refusal pass rate on an 8-message comparison set).

5. Five failures, root causes, fixes

1. Complaints not escalated (Fix 1). Case: complaint-01/03/04. The model apologised and offered to help but sometimes never called `escalate` ("Sure, I'll pass this to our team..." with `actions: []`), 0/3 baseline runs passing on complaint-04. Root cause: relying on the model to remember every time is fragile. Fix: a code-level safety net (`agent.py`) auto-calls `escalate` whenever a complaint/human-request intent (`data-driven`, `intents.py` + `lexicon.toml`) is detected and the model didn't call it itself this turn. Result: complaint pass rate 58.3% -> **100%** (`reports/runs/20260928-163524-fix5-6-7-combined/`).

2. Unknown items substituted (Fix 3), plus the arith-01 resolver regression it exposed. Case: unknown-01/03/04. "Do you make rabri?" answered with Kaju Katli's price as if it answered the question. Root cause: the prompt said to admit not knowing, but never forbade substituting a real, wrong product's price as a "helpful" answer. Fix: an explicit "NEVER substitute a different product's price or fact" rule with per-language examples. Result: unknown pass rate 27.8% -> **88.9%**. Verifying this fix's prompt changes exposed a *separate*, pre-existing bug: `arith-01` (100% at baseline) regressed to 0% -- traced directly to `pricing._resolve_item` picking the wrong SKU on a tie ("diwali gift box" vs "gift box large" matched by character length, not word specificity). Fixed by

rewriting alias matching to order-insensitive word-set comparison ranked by word count, plus a size-word disambiguation layer -- verified with 6 new unit tests; arith-01 restored (reports/runs/20260928-165509-fix-resolver-verify/).

3. lead-03 invented "Mr. Patel" (Fix 5). The model hallucinated a name, phone number and date **none of which the customer typed**, and never corrected it when the real customer's details arrived next turn. Root cause:

save_lead's arguments were trusted at face value. Fix: every name/phone/email argument must be grounded in text the customer actually wrote (whole-word match, casefold), plus a "lead nudge" that forces a save_lead attempt when the customer gives valid contact details and none has been saved yet. Result: lead pass rate 86.7% - > **100%** (final-2); the exact hallucination scenario is unit-tested (test_name_grounding_rejects_hallucinated_name).

4. hinglish-04: an invented order, caused by our own Fix 6. Fix 6 (forcing calculate_order whenever a "total" intent + quantity digit appears) fired on the bare number "7000" in a pure COD-limit question ("7000 ke order pe cash on delivery milega kya?"), and the model invented items ("Mixed Namkeen x10, Gulab Jamun x1") to justify a tool call the customer never asked for -- a regression Fix 6 itself introduced. Root cause: no check that a calculate_order call's items were things the customer actually asked for. Fix: item-argument grounding (reject any item not backed by a lexicon alias or literal SKU the customer wrote), plus narrowing the calc-nudge trigger to require a quantity attached to a product/unit word, not a bare number. Result: invented-amount rate dropped from the round-1 regression peak of 3.2% back to **1.4%** (final-2), with the exact scenario unit-tested (test_item_grounding_rejects_unmentioned_item, test_no_calc_nudge_for_bare_number).

5. hindi-06: "2 पैक" priced as kg (Fix 7 -> Fix 9c -> the tool-call-leak guard). "2 पैक चाहिए" (2 packs) was computed as unit="kg", giving ₹1,360 instead of ₹740. **The prompt-only fix failed:** clarifying the tool schema's unit description didn't change the model's argument choice (confirmed by direct tracing -- identical wrong result after the change). The actual fix was code-side: an independent pack-quantity extraction from the customer's own words, rejecting a calculate_order call whose unit disagrees (Fix 9c) -- verified by unit test (grand_total == 740) and live re-run (2/3, up from 0/3). **Honest final state:** hindi-06 remains flaky in the submission run (1/3) -- not from the pack/kg bug (confirmed absent in every trace checked), but from the model occasionally failing to call the tool at all, including leaking its own tool-call JSON as reply text (ordinal "calculate_order" को इस प्रकार संशोधित करें: { ... }). This new failure mode is what the Part 10A tool-call-leak guard (guards.py) targets -- a residual model-reliability limit on tool-call retry discipline, mitigated but not eliminated by a guard-and-retry layer around a small local model.

Manual testing on the chat page (a real multi-turn conversation, not the harness's per-case isolation) found further bugs the automated cases never exercised — a cross-item pack-quantity false positive breaking a 2-turn arithmetic case, and leads from two different people in one conversation merging into one record — both of which became new regression cases (long-session-01, lead-06) rather than one-off patches.

6. Cost and latency

Avg tokens in/out: 4954 / 111. Latency p50/p95: 2994 / 16932 ms. Cost: 0.00 INR / 100 conversations (local model, zero per-token cost).

INR cost is 0 for a local model (usd_per_1m*_tokens = 0 in config — Ollama has no per-token charge); the formula (tokens x \$/1M x INR/USD) is wired up for a future hosted-model swap. p95 latency (16932 ms) is well above p50 (2994 ms) because turns needing calculate_order plus a guard-triggered correction retry mean 3-4 sequential LLM calls on a local 7B model with no batching — the model_calls distribution is dominated by 1-call turns (simple Q&A) with a long tail of 3-4 call turns (arithmetic + retries).

7. One more week

Code-side quantity/unit extraction compared against every tool call (not just the pack/kg case Fix 9c already covers) to close the remaining argument-grounding gaps; making the model retry more reliably after a tool ERROR: instead of occasionally giving up into a fallback; streaming replies to the chat page; a human-approval queue for save_lead / escalate before they're considered final; persistent storage (a real DB) for leads/escalations/conversations; running the eval harness across several models on a schedule to catch model-swap regressions; and retrieval that scales to a much larger catalogue (proper embeddings/BM25 instead of a hand-maintained lexicon).